

Tareas Avanzadas en Desarrollo Web

1. El rol de un preloader en la experiencia de usuario Un preloader es básicamente ese indicador visual (un spinner, una barra de carga o una animación) que sale en pantalla mientras la página descarga los recursos pesados en segundo plano (como imágenes de alta resolución, scripts o datos de una API). Su mayor beneficio es psicológico: evita que el usuario piense que la página se rompió y se vaya (reduce la tasa de abandono). Además, disfraza el tiempo de espera real y evita el molesto efecto FOUC (*Flash of Unstyled Content*), que es cuando la web se va renderizando "por partes" y de forma fea antes de que carguen los estilos de CSS.

2. Carga de archivos con HTML y JavaScript El proceso empieza en el HTML con la etiqueta `<input type="file">`, que es la que abre la ventana del explorador de archivos del sistema. En JavaScript, nos colgamos de ese input escuchando el evento `change`. En cuanto el usuario elige un archivo, JS accede a él mediante un objeto llamado `FileList` (a través de `input.files[0]`). Este objeto nos da los metadatos (nombre, peso, tipo de archivo), pero para morder el contenido real y procesarlo en el navegador, necesitamos apoyarnos en la API de `FileReader`.

3. ¿Para qué sirve el objeto `FileReader`? Sirve como un puente que lee el contenido de los archivos de forma asíncrona directamente en la memoria del navegador, sin necesidad de subirlos primero a un servidor. Transforma el archivo crudo en formatos que el navegador entiende: un *Data URL* (Base64) si lo que quieres es mostrar una previsualización de una imagen, texto plano para leer archivos `.txt` o `.json`, o un *ArrayBuffer* si vas a meterte con datos binarios puros.

4. Ventajas de usar Flexbox hoy en día Flexbox resolvió los dolores de cabeza históricos de CSS. Sus ventajas clave son:

- Centrado sin trucos: Alinear elementos en vertical y horizontal ahora toma un par de líneas de código.
- Espaciado inteligente: Distribuye el espacio sobrante de forma proporcional entre los elementos de forma automática.
- Adiós a los anchos fijos: Los elementos se estiran o encogen según la pantalla para evitar desbordamientos.
- Orden flexible: Con la propiedad `order` puedes cambiar la posición visual de un elemento en pantalla sin tocar la estructura del HTML.

5. Maquetación fija vs. Responsiva La maquetación fija se quedó en el pasado; usa valores absolutos (como píxeles) y el diseño se rompe o exige hacer zoom horizontal en pantallas chicas. La maquetación responsiva, en cambio, se adapta como el agua. Usa unidades relativas (`%`, `rem`, `vw`), combinadas con *Media Queries* y herramientas como Flexbox o Grid, para que el diseño se

reacomode y fluya de forma óptima ya sea en un teléfono, una tablet o un monitor de 27 pulgadas.

6. Animaciones CSS y la regla `@keyframes` La regla `@keyframes` define la hoja de ruta de una animación. En ella declaras los estados de diseño por los que pasará un elemento usando porcentajes (de 0% a 100%) o las palabras `from` y `to`. Después, para aplicar esa animación a un elemento HTML, usas la propiedad `animation` en CSS, donde configuras el nombre del keyframe, los segundos que va a durar, la curva de velocidad (como `ease-in-out`) y si se va a repetir o no.

7. Animaciones y rendimiento web Para que una animación corra fluida a 60 FPS y no chupe la batería del usuario, la regla de oro es animar solo propiedades aceleradas por la tarjeta gráfica (GPU). Esto se reduce casi siempre a `transform` (para mover, escalar o rotar) y `opacity` (para desvanecimientos). Si te pones a animar propiedades como `width`, `margin` o `top`, obligas al navegador a recalcular el diseño de toda la página cuadro por cuadro (*Reflow*), lo que genera tirones visuales (*jank*).

8. Previsualizar una imagen antes de subirla al servidor El flujo estándar en el navegador es el siguiente:

1. El usuario selecciona la foto en el `<input type="file">`.
2. JS detecta el cambio (`change`) y agarra el archivo desde `event.target.files[0]`.
3. Es buena práctica validar rápido el tipo de archivo (MIME tipo `image/*`) y que no pese demasiado.
4. Para mostrarla, generas una ruta temporal usando `URL.createObjectURL(file)` (que es lo más rápido) o lees el archivo con `FileReader` para sacar su Base64.
5. Esa ruta o cadena se le clava directamente al atributo `src` de una etiqueta `<img>` en el HTML, y la foto aparece en pantalla al instante de forma local.

9. Carga dinámica y animaciones en la experiencia de usuario Estas herramientas son las que hacen que una web deje de sentirse como un documento estático y se note como una aplicación nativa. La carga dinámica (con Fetch/AJAX) actualiza trozos de la pantalla sin recargar todo el sitio, haciendo la navegación instantánea. Las animaciones (o micro-interacciones) le dan *feedback* al usuario: un botón que reacciona al pasar el cursor, un esqueleto de carga que avisa que los datos vienen en camino, o un mensaje de éxito que aparece suavemente. Todo esto reduce la fricción y hace la interfaz intuitiva.

Propuesta: "MedicalScan AI" (Analizador de Radiografías)

El Concepto: Una plataforma web optimizada para que el personal médico pueda cargar radiografías y obtener un pre-diagnóstico asistido por Inteligencia Artificial en segundos.

Integración de tecnologías en el sistema:

- Estructura y Adaptabilidad (Grid + Flexbox): El layout principal se apoya en CSS Grid para maquetar la pantalla en dos paneles en computadoras de escritorio (zona de carga a la izquierda y resultados a la derecha). El contenido interno de los paneles se organiza con Flexbox para mantener todo alineado. En móviles, las *Media Queries* apilan todo en una sola columna para que el médico pueda usar la herramienta en sus rondas desde una tablet o teléfono sin perder espacio de lectura.
- Carga Local y Confirmación Visual (FileReader): La zona de arrastrar y soltar oculta el input de archivos. Cuando el médico suelta la radiografía, JavaScript intercepta el archivo y usa la API de FileReader para generar una vista previa de la imagen en alta definición de forma inmediata. Esto funciona como un paso de seguridad crítico: el médico puede verificar visualmente que cargó la placa correcta antes de enviarla a procesar.
- Gestión de Espera (Preloader): Al dar clic en "Analizar", la imagen se opaca y se activa un preloader en pantalla completa (por ejemplo, una línea láser que simula escanear la placa). Dado que el análisis por IA en el servidor toma unos segundos, este indicador es vital para que el usuario sepa que el sistema está trabajando y no intente dar clics repetidos por desesperación.
- Feedback e Impacto Visual (@keyframes): El láser del preloader se mueve de arriba a abajo usando una animación CSS basada en @keyframes optimizada con `transform: translateY()` para no consumir recursos del procesador. Cuando la API responde, el preloader se desvanece suavemente con `opacity` y las tarjetas con los resultados médicos entran de forma escalonada flotando desde el fondo. Si la IA detecta una anomalía de alta prioridad (como una fractura expuesta), la tarjeta del reporte activa una animación infinita con un pulso rojo sutil para captar de inmediato la atención del especialista.